

Guide Git pour Projet C++ avec CMake et Visual Studio

1 Introduction

Ce document présente un workflow simple pour utiliser **Git** dans un projet C++ utilisant **CMake** et **Visual Studio**. L'objectif est d'éviter les conflits inutiles et de maintenir un dépôt propre.

2 Configuration initiale

Configurer Git pour utiliser automatiquement le rebase lors d'un pull.

```
git config --global pull.rebase true
```

Effet :

Lors d'un pull, Git rejoue vos commits au-dessus de la branche distante.

Avant :

```
origin/enzo-test
A -- B -- C

local
A -- B -- D
```

Après pull :

```
A -- B -- C -- D
```

3 Workflow recommandé

3.1 1. Se mettre à jour avant de travailler

```
git checkout enzo-test
git pull
```

git checkout change la branche active.

git pull effectue :

- git fetch : récupère les commits distants
- git rebase : rejoue vos commits au-dessus

3.2 2. Modifier le code

Travaillez normalement dans votre projet.

3.3 3. Ajouter les modifications

```
git add .
```

Les fichiers passent dans la **staging area**.

3.4 4. Créer un commit

```
git commit -m "Add webcam capture"
```

Un commit est un **snapshot du projet**.

3.5 5. Se resynchroniser avant de push

```
git pull
```

Cela permet d'éviter les conflits si quelqu'un a modifié la branche distante.

3.6 6. Envoyer les modifications

```
git push
```

Les commits sont envoyés vers le dépôt distant (ex : GitHub).

4 Cycle de travail complet

```
git pull
git add .
git commit -m "message"
git push
```

5 Fichiers à ignorer

Les fichiers générés ne doivent jamais être versionnés.

Exemple de `.gitignore` :

```
.vs/
.vscode/
build/
out/
CMakeFiles/
CMakeCache.txt
*.exe
*.obj
*.dll
*.lib
```

Ces fichiers sont générés par CMake ou Visual Studio.

6 Commande utile

6.1 Vérifier l'état du dépôt

```
git status
```

Permet de voir :

- la branche actuelle
- les fichiers modifiés
- les fichiers prêts à être commit
- les fichiers non suivis

7 Fonctionnement simplifié de Git

```
working directory
|
v
git add
|
v
staging area
|
v
git commit
|
v
history
|
v
git push
|
v
remote
```

8 Conclusion

Pour éviter les problèmes avec Git :

- Toujours faire un **git pull** avant de commencer à travailler
- Ne jamais versionner les fichiers de build
- Faire des commits petits et fréquents